

Code-Konventionen — BANDspirit Backend

CC-M7: Dieses Dokument definiert Namens-, Sprach- und Strukturkonventionen für das Backend.

1. Namenskonventionen

1.1 Sprache

Grundregel: Deutscher **Fachcode** (Domänenmodelle, Entitäten, Services) + englische **Infrastruktur** (ASP.NET-Begriffe, generische Utilities).

Element	Sprache	Beispiel
Entitäten (Domain)	Deutsch	Firma , Kreis , Benutzer-Rolle , S3Circle
Eigenschaften (Domain)	Deutsch	Name , Beschreibung , Aktiv , Veröffentlichungsdatum
Services (Domain)	Deutsch + Suffix	RbacService , AuthService , NotificationService
Controller	Deutsch + Suffix	CirclesController , BenutzerRollenController
DTOs	Englisch + Suffix	LoginRequest , UserDto , ODataResponse<T>
Infrastruktur	Englisch	BandSpiritDbContext , PermissionAuthorizationHandler , SecurityHeadersMiddleware
Konfiguration	Englisch	JwtOptions , appsettings.json

Sonderfall: S3-Modelle (Sociocracy 3.0) verwenden oft **englische Originalbegriffe** (S3Circle , S3Role , S3Proposal , Driver , Objection), um die Nähe zur S3-Literatur zu wahren; deutsche Eigenschaften bleiben deutsch (Beschreibung , Titel).

1.2 Berechtigungen (Permissions)

Format: <bereich>:<ressource>:<aktion> oder <bereich>:<aktion>

Beispiele:

- user:read , user:create , user:delete , user:manage
- org:circle:read , org:circle:create , org:role:assign
- dashboard:read , stammdaten:manage

Zentrale Verwaltung: Alle Permissions sind als Konstanten in `Infrastructure/Auth/Permissions.cs` definiert.

Verwendung: In Controllern IMMER die Konstanten nutzen, niemals String-Literale:

```
[Authorize(Policy = Permissions.UserRead)] // ✓
[Authorize(Policy = "user:read")]         // ✗
```

1.3 Rollen

Standardrollen: `Admin`, `User`

Weitere Rollen werden in der Entität `BenutzerRolle` als Stammdaten gepflegt. Rollennamen sind **PascalCase**, z. B. `Admin`, `Geschäftsleitung`, `Mitarbeitende`.

2. Architekturkonventionen

2.1 Autorisierung (RBAC)

Reihenfolge:

1. Cache (`IMemoryCache`, 60 Sek.)
2. Datenbank (`RolePermission`-Tabelle)
3. Hartkodierter Fallback (nur Notfall; Admin = alle Rechte, andere = lesende Grundrechte)

Source of Truth: Das Seeding (`RolePermissionSeeder`) definiert initial Rollen + Berechtigungen. Der Fallback dient nur zur Absicherung bei DB-Problemen.

2.2 Audit-Logging

CC-M2: Konsolidierte Architektur (Stand nach Korrektur37):

Quelle	Zweck	Wann
DbContext.SaveChanges	Fachliche Datenänderungen (vorher/nachher)	CREATE/UPDATE/DELETE von Entitäten
AuditMiddleware	Zugriffs-/Sicherheitseignisse	401/403/≥400-Responses, IP-Logging

AuditService: Wird als gemeinsame Schnittstelle vom `DbContext` und der Middleware genutzt, um `AppLog`-Einträge zu schreiben.

Ziel: Keine Doppel-Einträge — `DbContext` loggt nur Änderungen, Middleware nur Zugriffe/Fehler.

JSON-Schema für Audit-Detail

Feld `AppLog.Details` (JSON):

```
{
  "vorher": { "Name": "Alt", "Aktiv": true },
  "nachher": { "Name": "Neu", "Aktiv": false },
  "geaenderteFelder": ["Name", "Aktiv"],
  "ip": "192.168.1.10",
  "statusCode": 200,
  "pfad": "/odata/Users(123)"
}
```

Schlüssel:

- vorher / nachher : Feldwerte vor/nach der Änderung (CREATE/UPDATE/DELETE)
- geaenderteFelder : Liste der geänderten Eigenschaften
- ip : IP-Adresse des Aufrufers (bei Middleware-Logs)
- statusCode : HTTP-Statuscode (bei Middleware-Logs)
- pfad : Request-Pfad

3. Konfigurationskonventionen

3.1 Typisierte Konfiguration (IOptions)

Grundregel: Konfigurationsabschnitte werden als typisierte Klassen unter `Infrastructure/Config/` abgebildet und über `IOptions<T>` injiziert — **NICHT** direkter Zugriff auf `IConfiguration`.

Beispiel: `JwtOptions` (CC-M4)

```
public class JwtOptions
{
    public const string Section = "Jwt";
    public string Secret { get; set; } = string.Empty;
    public string Issuer { get; set; } = "BandSpirit";
    public string Audience { get; set; } = "BandSpirit";
    public int ExpiresInMinutes { get; set; } = 480;
}

// Registrierung (Program.cs)
builder.Services.Configure<JwtOptions>(config.GetSection(JwtOptions.Section));

// Nutzung (Service)
public JwtTokenService(IOptions<JwtOptions> jwtOptions)
{
    _jwtOptions = jwtOptions.Value;
}
```

3.2 Secrets

Produktiv: Geheimnisse (DB-Passwort, JWT-Secret, SMTP, S3-Keys) kommen **NUR** aus:

- Umgebungsvariablen
- Azure Key Vault / Secret-Store

NIE im Repository (`appsettings.json` enthält nur Platzhalter).

4. Testing-Konventionen

Projekt: `BandSpirit.Api.Tests` (xUnit)

Fokus:

- Authentifizierung (`AuthService` , `JwtTokenService`)
- Autorisierung (`RbacService` , `PermissionAuthorizationHandler`)
- Audit-Erfassung (`DbContext` , `Middleware`)

Struktur:

```
Tests/
  Auth/
    AuthServiceTests.cs
    JwtTokenServiceTests.cs
  Authorization/
    RbacServiceTests.cs
  Audit/
    AuditMiddlewareTests.cs
```

Namenskonvention: `<Klasse>Tests.cs` , Methoden: `<Methode>_<Szenario>_<Erwartung>`

Beispiel: `Login_ValidCredentials_ReturnsToken()`

5. OData-Konventionen

5.1 Serialisierung

Wichtig: `odataModelBuilder.EnableLowerCamelCase()` ist aktiv — OData-Antworten liefern Eigenschaften in **camelCase** (`name` , `email` , `createdAt`), NICHT PascalCase.

Frontend-Erwartung: Das Frontend erwartet camelCase. Enum-Werte bleiben PascalCase (z. B. `"Admin"`).

5.2 DTOs für sensible Entitäten

User-Entität: Wird als `UserDto` (ohne `Password` -Feld) über OData exponiert. Der `UserController` gibt **niemals** die `User` -Entität direkt zurück.

6. Versionierung & Breaking Changes

- **Migrationen:** EF Core Migrations unter `Infrastructure/Data/Migrations/` .
 - **Breaking API-Changes:** Versionierung über URL-Pfade (`/api/v2/...`) oder OData-Versionierung.
 - **Audit-Detail-Schema:** Änderungen am JSON-Schema dokumentieren (`CONVENTIONS.md` + Versionsnummer).
-

Letzte Aktualisierung: Korrektur37 (mittelfristige Maßnahmen CC-M1...M7)